



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Name of Student: Aryan Singh Kasera

PRN: 25070122052

Subject: Programming with Java

Sem: III AY: 2026-27

Assignment No: 10

TITLE: Write a Java program to demonstrate the use of abstract classes and abstract methods

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Theory

Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

1.Create an abstract Payment class and implement Credit Card and UPI payment classes.?

Main.java

Share

Run

```
1 ▾ abstract class Payment {
2     abstract void pay(double amount);
3 }
4
5 ▾ class CreditCard extends Payment {
6     void pay(double amount) {
7         System.out.println("Credit Card Payment: " + amount);
8     }
9 }
10
11 ▾ class UPI extends Payment {
12     void pay(double amount) {
13         System.out.println("UPI Payment: " + amount);
14     }
15 }
16
17 ▾ class Main {
18     public static void main(String[] args) {
19         Payment p1 = new CreditCard();
20         Payment p2 = new UPI();
21
22         p1.pay(2500);
23         p2.pay(1500);
24     }
25 }
```

OUTPUT

Output

```
Credit Card Payment: 2500.0
UPI Payment: 1500.0

=== Code Execution Successful ===
```

2. Create an abstract FoodOrder class with an abstract method calculateBill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.

Main.java    Share 

```
1 abstract class FoodOrder {
2     double amount;
3
4     FoodOrder(double amount) {
5         this.amount = amount;
6     }
7
8     abstract void calculateBill();
9 }
10
11 class DineInOrder extends FoodOrder {
12     DineInOrder(double amount) {
13         super(amount);
14     }
15
16     void calculateBill() {
17         double total = amount + (amount * 0.10);
18         System.out.println("Dine-In Bill: " + total);
19     }
20 }
21
22 class TakeAwayOrder extends FoodOrder {
23     TakeAwayOrder(double amount) {
24         super(amount);
25     }
26
27     void calculateBill() {
28         double total = amount + 30;
29         System.out.println("Takeaway Bill: " + total);
30     }
31 }
32
33 class Main {
34     public static void main(String[] args) {
35         FoodOrder f1 = new DineInOrder(900);
36         FoodOrder f2 = new TakeAwayOrder(800);
37
38         f1.calculateBill();
39         f2.calculateBill();
40     }
41 }
```

OUTPUT

Output

Dine-In Bill: 990.0

Takeaway Bill: 830.0

=== Code Execution Successful ===

Conclusion:

This program demonstrates the use of abstract classes and abstract methods in Java. It shows how abstraction hides implementation details while allowing subclasses to provide their own implementations. This improves code reusability, flexibility, and maintainability in object-oriented programming.